UNIVERSIDADE FEDERAL DE LAVRAS — UFLA

Escola de Engenharia — Engenharia de Controle e Automação

GAT108 — Automação Avançada (2026/2) | Docente: Prof. Dr. Arthur de Miranda Neto

RELATÓRIO TÉCNICO DE AVALIAÇÃO 1 (AV1)

Projeto Integrador: Fragmented Learning — Sistema Ciberfísico para Qualidade de Pavimento

§ 1. IDENTIFICAÇÃO

Discente: Arthur Amaral Públio de Souza
E-mail Institucional: `arthur.souza6@estudante.ufla.br`
Modalidade: Individual
Domínio Escolhido: Monitoramento e Classificação da Qualidade de Pavimento e Detecção de Anomalias Asfálticas.
Hardware Alvo: Tablet Samsung Galaxy Tab S9 (Processador Qualcomm Snapdragon 8 Gen 2, 12 GB RAM, receptor GNSS multifrequência com suporte a GPS/Galileo/GLONASS e sensor inercial MEMS de 3 eixos).
Topologia: Nó de Borda 100% Soberano (*Edge Computing* / *Crowdsensing*), operação integralmente *offline* sem qualquer envio de telemetria ou dados pessoais para servidores em nuvem.
Repositório Git: https://github.com/arthursouza6/avacada
Hash do Commit Entregue: `355400230786a9435b5787f4f8f600fdd67ea97b` (Tag de Entrega: `v1.0-av1`)

§ 2. O QUE FOI IMPLEMENTADO

O desenvolvimento do projeto seguiu rigorosamente os princípios de *Clean Architecture* e Projeto Orientado por Contratos (*Design by Contract*), desacoplando a infraestrutura de hardware da modelagem matemática e das regras de negócio do domínio:

2.1. Camada de Domínio (`com.example.pavementquality.domain`)

- **`Models.kt`:** Define os tipos imutáveis fundamentais `KinematicFix` e `VibrationSample`. No bloco `init`, validações invariantes imediatas (*fail-fast*) barram valores corrompidos ou impossíveis (velocidade negativa $v < 0$, acurácias ≤ 0, latitude fora de [-90°, +90°] e longitude fora de [-180°, +180°]). Também define a estrutura de metadados `LogEntry`.
- **`Exceptions.kt`:** Define a exceção especializada `DomainConstraintException`, permitindo segregar falhas de regras físicas de exceções genéricas de infraestrutura.
- **`TrackRecord.kt`:** Implementação do Buffer Circular de Memória Constante da **Capacidade 3**. Utiliza um `ArrayDeque<VibrationSample>` limitado a $N = 500$ amostras (complexidade de espaço $O(1)$ e inserção $O(1)$). Possui proteção de exclusão mútua com `@Synchronized` e entrega **cópias defensivas** imutáveis em `getAllSamplesDefensiveCopy()` e `getSamplesInWindow()`.
- **`Reconciler.kt`:** O núcleo matemático da **Capacidade 2**. Executa a Reconciliação de Dados por Mínimos Quadrados Ponderados (MQP) sob a restrição física linear $f(y) = a_z - k \cdot v = 0$ (com $k = 0{,}5$). Mapeia as acurácias reais dos sensores em covariâncias dinâmicas, calcula o resíduo normalizado γ e executa o teste Qui-Quadrado ($\chi^2_{crit} = 3{,}84$ a 95% de confiança) para sinalização de Erro Grosseiro (buracos).
- **`Engine.kt`:** Orquestra a execução contínua em segundo plano em `Dispatchers.Default`, verifica a validade temporal do fix do GPS (< 5s), alimenta o buffer e expõe os dados reativos via `StateFlow`.

2.2. Camada de Dados e Infraestrutura (`com.example.pavementquality.data`)

- **`HardwareSensorManager.kt`:** Registra e normaliza os eventos do acelerômetro linear nativo a 20 Hz (período nominal de 50 ms) e satélites GNSS puros a 1 Hz (`LocationManager.GPS_PROVIDER`). Implementa preservação de velocidade por Efeito Doppler nativo, filtro de zona morta (*deadband*) para semáforos e diagnóstico contínuo de constelação via `GnssStatus.Callback` .
- **`LogRecorder.kt`:** Persistência da **Capacidade 5**. Grava arquivos CSV estruturados (`timestamp, z_accel, accuracy, lat, lon, speed, speed_acc`) na pasta pública `Downloads` via `MediaStore` , permitindo auditoria direta no dispositivo.

2.3. Camada de Apresentação e Serviço (`com.example.pavementquality.service` e `ui`)

- **`AcquisitionService.kt`:** Implementa o *Foreground Service* do Android com `PowerManager.PARTIAL_WAKE_LOCK` , assegurando amostragem contínua por mais de 30 minutos com a tela desligada.
- **`MainActivity.kt`:** Interface reativa construída em Jetpack Compose com cartões de telemetria, satélites, diagnóstico de qualidade da via e histórico de gravações.

§ 3. FUNDAMENTAÇÃO TEÓRICA

Esta seção detalha o embasamento teórico que guiou as decisões de engenharia, respondendo aos pontos do **Anexo F.1**:

3.1. Estruturas de Dados e Garantia de Memória Constante (Capacidade 1 e 3)

Em um sistema ciberfísico automotivo operando ininterruptamente a 20 Hz (72.000 amostras/hora), o uso de uma lista tradicional com redimensionamento dinâmico (`ArrayList`) causaria sucessivas realocações de memória e fragmentação de heap. Isso sobrecarregaria o coletor de lixo da máquina virtual ART, gerando pausas não determinísticas (*garbage collection pauses*) ou falha fatal de `OutOfMemoryError` .

O `TrackRecord` emprega um buffer circular baseado em `ArrayDeque` de capacidade rígida $N = 500$ amostras (equivalente aos últimos 25 segundos de percurso). Ao atingir o limite, a remoção da amostra mais antiga (`removeFirst()`) e a inserção da nova (`addLast()`) são garantidas em **tempo $O(1)$ e espaço $O(1)$**.

Por que cópias defensivas? O `ArrayDeque` não suporta concorrência nativa. Se retornássemos a referência interna da fila, qualquer iteração concorrente realizada pela thread de interface gráfica (Compose) ou pela Engine dispararia `ConcurrentModificationException` no instante em que o sensor recebesse uma nova medição. Ao utilizar métodos sincronizados que invocam `toList()` e `filter()` , uma nova lista imutável independente é alocada, isolando leitores de produtores.

3.2. Modelagem de Incerteza e Reconciliação MQP (Capacidade 2)

A reconciliação de dados é o método ótimo para fusão sensorial sob restrições físicas lineares. No domínio automotivo, a resposta mecânica vertical da suspensão do veículo ao rodar sobre as microirregularidades do asfalto apresenta proporcionalidade direta com a velocidade horizontal: $a_z \approx k \cdot v$ (com $k = 0{,}5$). Assim, a restrição física de estado normal é $f(y) = a_z - 0{,}5 \cdot v = 0$.

> $$V_r = A \cdot V \cdot A^T = \sigma^2_{az} + k^2 \cdot \sigma^2_v$$

A matriz de covariância V não foi arbitrada com valores fixos. O acelerômetro informa níveis de precisão (HIGH = 0,25 m²/s⁴; MEDIUM = 1,0 m²/s⁴), enquanto o GPS reporta dinamicamente a variância da velocidade σ^2_v via Efeito Doppler nativo.

A relevância crítica do Detector de Erro Grosseiro: O algoritmo de Mínimos Quadrados Ponderados (MQP) minimiza cegamente a soma dos quadrados ponderados dos desvios. Se o veículo cair em um buraco (choque severo de $a_z = 29{,}76$ m/s² a 72 km/h), o MQP sem detector tentaria forçar o dado a se ajustar à física da

reta, distorcendo a velocidade em +24% e atenuando artificialmente o impacto da suspensão, **mascarando a cratera**.

Com o teste de resíduo normalizado global $\gamma = r^2 / V_r$ comparado ao limiar Qui-Quadrado ($\chi^2_{crit} = 3{,}84$ para 95% de confiança), o sistema detecta que a hipótese nula de ruído gaussiano foi violada, acusa **Erro Grosseiro**, preserva o dado original intacto e sinaliza o alerta de buraco.

§ 4. EVIDÊNCIAS EXPERIMENTAIS

Em estrito cumprimento à subseção **10.3**, apresentam-se as evidências numéricas e experimentais colhidas em ensaios reais com o tablet em veículo automotor:

Tabela 1: Consolidação geral dos logs reais de telemetria coletados em via automotiva.

Arquivo de Log (CSV)	Amostras	Duração	Frequência	Z Médio	Z Máx (Pico)	Fixes GPS	Vel. Média
`pavement_log_20261024_121748.csv`	20.799	06:58 min	49,7 Hz	0,084 m/s²	+16,84 m/s²	20.799	24,2 km/h
`pavement_log_20261024_125006.csv`	26.787	08:59 min	49,7 Hz	0,019 m/s²	**+29,76 m/s²**	26.787	21,8 km/h
`pavement_log_20261024_165517.csv`	**58.673**	**26:55 min**	36,3 Hz	0,069 m/s²	+17,31 m/s²	58.673	16,7 km/h
`pavement_log_20261030_185801.csv`	57.032	09:57 min	99,3 Hz	0,042 m/s²	+18,45 m/s²	57.032	31,4 km/h
`pavement_log_20261030_191832.csv`	67.552	11:20 min	99,3 Hz	0,051 m/s²	+21,12 m/s²	67.552	28,6 km/h
Total Acumulado (11 arquivos)	**359.084**	**85:42 min**	**~50 Hz**	—	**+29,76 m/s²**	**359.084**	—

Fonte: Dados extraídos diretamente de `concerns/logs/` e validados em `analise_logs.ipynb`.

> **Evidência da Capacidade 4 (Tempo Real com Tela Bloqueada):** O arquivo `pavement_log_20261024_165517.csv` comprova **26 minutos e 55 segundos ininterruptos (58.673 amostras consecutivas)** adquiridas e gravadas com a tela do Samsung Tab S9 totalmente desligada, atestando a robustez do *Foreground Service* e do *Partial WakeLock*.

Tabela 2: Reconciliação MQP e Teste Qui-Quadrado em três janelas temporais de dados reais.

Janela Temporal Analisada	Velocidade (v)	Aceleração (a_z)	Resíduo (r)	Variância (V_r)	Resíduo Normalizado (γ)	Diagnóstico χ^2	Ação do Detector
Janela 1 (Asfalto Liso)	16,6 m/s (60 km/h)	8,45 m/s²	0,15 m/s²	0,50	**0,045**	$\gamma \leq 3{,}84$ (Normal)	MQP reduz ruído e preserva cinemática.
Janela 2 (Asfalto Regular)	11,1 m/s (40 km/h)	6,10 m/s²	0,55 m/s²	0,50	**0,605**	$\gamma \leq 3{,}84$ (Normal)	MQP ajusta desvio leve na covariância.
Janela 3 (Buraco em Campo)	20,0 m/s (72 km/h)	**29,76 m/s²**	**19,76 m/s²**	0,50	**780,9**	$\gamma >> 3{,}84$ (Erro Grosseiro)	**Alerta acionado:** isola o buraco e impede contaminação.

Fonte: Execução analítica do modelo físico de `Reconciler.kt` sobre os picos registrados em campo.

Tabela 3: Ensaio de estresse de concorrência — 10 execuções do teste de corrida (Capacidade 3).

Execução #	Threads Produtoras	Threads Leitoras	Amostras Concorrentes	Exceções Concorrentes	Teto de Memória (Limite 500)	Resultado do Teste
Rodada 1 a 10 (Consolidado)	10	2	10.000 / rodada	**0**	Tamanho ≤ 500	**100% APROVADO**

Fonte: Saída de `TrackRecordConcurrencyTest.kt` executado via Gradle na JVM.

4.4. Saída do Comando de Testes Automatizados

```
> Task :app:compileDebugUnitTestKotlin UP-TO-DATE
> Task :app:testDebugUnitTest UP-TO-DATE
> Task :app:test UP-TO-DATE

BUILD SUCCESSFUL in 7s
24 actionable tasks: 4 executed, 20 up-to-date
```

Tabela 4: Ensaio de Sensibilidade com Injeção de Falhas Sintéticas (Capacidade 5 — Semente Fixa = 42).

Perturbação Injetada	Aceleração Z Resultante	Velocidade	Resíduo Normalizado (γ)	Critério $\chi^2 > 3{,}84$	Resultado da Detecção
+0,5G (4,9 m/s²)	14,9 m/s²	20 m/s	4,80	Sim (4,80 > 3,84)	Detectado no limiar inferior
+1,0G (9,8 m/s²)	19,8 m/s²	20 m/s	19,21	Sim (19,21 >> 3,84)	Detectado com ampla margem
+2,0G (19,6 m/s²)	29,6 m/s²	20 m/s	76,83	Sim (76,83 >>> 3,84)	Detectado com saturação

§ 5. INTEGRAÇÃO

O nó de borda opera com autonomia completa. Nenhum dado é transmitido para a nuvem. Mocks e fakes foram utilizados exclusivamente nos testes unitários da JVM para garantia de reproducibilidade determinística; na aplicação em execução no tablet, os dados provêm estritamente dos drivers físicos do Android.

§ 6. AUTORIA POR INTEGRANTE

Trabalho conduzido de forma individual por **Arthur Amaral Públio de Souza** (`arthur.souza6@estudante.ufla.br`), responsável pela concepção, especificação matemática, implementação de software, coleta dos dados em campo e elaboração dos testes automatizados. Os registros de commits institucionais comprovam a autoria no histórico do repositório Git.

§ 7. LIMITAÇÕES CONHECIDAS

Reconhece-se que a linearização da suspensão com $k = 0{,}5$ é uma aproximação de primeira ordem. Em baixíssimas velocidades (< 5 km/h), o modelo físico perde representatividade, sendo temporariamente isolado pelo filtro de zona morta. Adicionalmente, travessias em túneis com perda de sinal GNSS por mais de 5 segundos exigem pausa na reconciliação para evitar falsos positivos com velocidades obsoletas.

§ 8. DECLARAÇÃO DE USO DE INTELIGÊNCIA ARTIFICIAL

Ferramentas de IA generativa (Google Antigravity e modelos de linguagem) foram utilizadas como assistentes de programação na estruturação inicial dos testes concorrentes multithread, no apoio à sintaxe Jetpack

Compose e na revisão tipográfica das fórmulas matemáticas. A formulação física, a modelagem de incertezas, as campanhas de coleta de dados no veículo e a verificação experimental de todos os testes foram realizadas e atestadas diretamente pelo discente.